

PROGRAMACIÓN AVANZADA

Sesión 21

Archivos de texto



NOTAS DEL
PROFESOR

Índice

1	Objetivos de la sesión	2
2	Prerrequisitos	2
3	Arranque y preguntas de sondeo	3
4	Memoria vs. archivos	3
5	¿Qué es un archivo? Demo con hex editor	3
6	La biblioteca <fstream>	4
7	Demo en vivo: los tres ejemplos	5
7.1	archivos1.cpp: leer línea por línea	5
7.2	archivos2.cpp: escribir en un archivo	5
7.3	archivos3.cpp: leer valores numéricos	6
8	Actividad: agenda de cumpleaños	7
9	Errores frecuentes	10
10	Soluciones a los ejercicios	10
11	Rúbrica de la actividad	13
12	Conexiones con temas posteriores	13
13	Materiales y recursos	13

Objetivos de la sesión

Al finalizar la sesión el alumno será capaz de:

1. Explicar la diferencia entre almacenamiento en memoria RAM y en archivo de disco, y cuándo conviene cada uno.
2. Crear, leer y escribir archivos de texto en C++ usando `ifstream` y `ofstream`.
3. Usar `getline` para leer líneas completas y `>>` para leer tokens numéricos de un archivo.
4. Escribir un programa que genere un archivo de datos estructurado, y otro que lo consuma para responder consultas.
5. Participar en una actividad colaborativa donde los datos de varios programas se integran en un solo archivo.

10 min	Gancho histórico + preguntas de arranque
10 min	Símil del escritorio; tabla RAM vs. disco
15 min	¿Qué es un archivo? Demo con editor hex
5 min	Presentar <code><fstream></code> y las tres clases
25 min	Demo en vivo: <code>archivos1</code> , <code>archivos2</code> , <code>archivos3</code>
35 min	Actividad: agenda de cumpleaños (Parte A + Parte B)

Prerrequisitos

- `cin` y `cout` con `>>` y `<<` (sesiones 1–2).
- `string` y `getline` desde teclado (sesión 2).
- Tabla ASCII y representación de caracteres (sesión 3).
- Compilación básica con `g++` y ejecución desde terminal.

Esta sesión no requiere apuntadores, structs ni arreglos avanzados. La actividad final sí puede llegar a necesitar arreglos simples o vectores para almacenar los datos de la agenda, pero los alumnos tienen esa base de sesiones anteriores.

Señal de alerta: si algún alumno no recuerda la diferencia entre `cin >>` y `getline`, detenerse 3 minutos a recordarlo con un ejemplo en pantalla antes de abrir archivos. El modelo mental de “el buffer del teclado” es el mismo que el de “el buffer del archivo”.

Arranque y preguntas de sondeo

1. “¿Qué le pasa a una variable cuando el programa termina?”

Respuesta esperada: desaparece / se pierde. Seguir con: “¿y si quieres que ese valor esté disponible la próxima vez que ejecutes el programa?” Dejar que propongan soluciones (variable global, retornar el valor, etc.) antes de decir: “la solución real se llama archivo.”

2. “¿Cuántas veces abrieron hoy un archivo antes de llegar aquí?”

Respuesta esperada: muchas (WhatsApp, fotos, música, documentos). Seguir con: “todos esos archivos son secuencias de bytes. Hoy van a crear sus propios archivos desde C++.”

3. “¿Por qué creen que la carpeta del explorador de archivos se llama *carpeta*?”

Respuesta esperada: porque se parece a una carpeta real. Usar esto para introducir la historia de Xerox PARC y el origen deliberado de la metáfora. Es el gancho para la sección histórica.

Memoria vs. archivos

El símil del escritorio funciona bien si se actúa físicamente: señalar el escritorio (o la pantalla), abrir el cajón del escritorio real si hay uno. Preguntar: “¿qué pasa si empujas todos los papeles del escritorio al piso?” Eso es lo que hace el sistema operativo cuando el programa termina: limpia la memoria. “¿Y el cajón?” Los cajones se quedan como están.

Después mostrar la tabla comparativa y pedir a un alumno que la lea en voz alta. Preguntar: “¿cuándo prefieren usar memoria? ¿cuándo un archivo?” Respuesta esperada: memoria para cálculos en curso, archivo para guardar resultados permanentes.

¿Qué es un archivo? Demo con hex editor

Esta demo es uno de los momentos más memorables de la sesión. El objetivo es que los alumnos *vean* que un archivo es solo bytes, no magia.

1. Crear un archivo de texto vacío:

- Windows: clic derecho → Nuevo → Documento de texto.
- Linux: `touch prueba.txt` en terminal.

Mostrar el tamaño: **0 bytes**. Preguntar: “¿qué esperaban?”

2. Abrir con el editor de texto, escribir `H` y guardar. Mostrar el tamaño: **1 byte**. Preguntar: “¿cuántos bytes esperaban?”
3. Abrir `prueba.txt` en `hexed.it` (o cualquier hex editor en línea). Mostrar el byte `0x48` y conectarlo con la tabla ASCII de sesiones anteriores.
4. Escribir `Hola` completo. En el hex editor: cuatro bytes, `48 6F 6C 61`. Pedir a algún alumno que identifique cada carácter.
5. Agregar un salto de línea con Enter y guardar. En Linux aparece `0A`; en Windows aparece `0D 0A`. Explicar que `0A` es LF (Line Feed) y `0D` es CR (Carriage Return) — herencia de las máquinas de escribir donde había que hacer dos acciones separadas para pasar de línea.
6. Preguntar: “¿qué diferencia hay entre este archivo y una imagen JPEG?” Respuesta: ninguna en fondo — también son bytes. Lo que cambia es cómo el programa los interpreta.

Por qué vale la pena hacer esto: muchos alumnos creen que un archivo de texto es algo cualitativamente distinto a un archivo binario. Ver los bytes directamente rompe esa idea y prepara la sesión 22 sobre archivos binarios.

¿Por qué el archivo vacío tiene 0 bytes y no algún encabezado?

En sistemas Unix y la mayoría de sistemas modernos, un archivo de texto plano (`.txt`) no tiene encabezado. Empieza directamente con los datos. Otros formatos como JPEG, PDF o DOCX sí tienen encabezados (*magic bytes*) que identifican el tipo de archivo. Los primeros 2 bytes de cualquier JPEG son siempre `FF D8`.

¿`sizeof(char)` puede ser distinto de 1?

No. El estándar C++ garantiza `sizeof(char) == 1` por definición. Otras preguntas sobre el tamaño de los tipos son válidas (`int` puede ser 2 o 4 bytes según la plataforma), pero `char` siempre es 1.

¿El problema CRLF vs LF les va a afectar en este curso?

Raramente, porque `<fstream>` en modo texto hace la conversión automáticamente al leer y al escribir. El problema surge cuando se abre el archivo en modo binario o cuando se comparte entre sistemas sin conversión. Para la actividad de hoy no es un problema.

La biblioteca <fstream>

Al mostrar la tabla de las tres clases, enfatizar la simetría con `cin / cout` :

- `cin` es un `istream` conectado al teclado.
- `ifstream` es un `istream` conectado a un archivo.

- La diferencia es solo dónde vienen los datos; los operadores son idénticos.

Esto reduce la carga cognitiva: los alumnos ya saben usar `>>` y `<<`. Solo aprenden a apuntar esos operadores a un archivo.

Demo en vivo: los tres ejemplos

- **archivos1.cpp: leer línea por línea**

1. Mostrar `canciones.txt` con `cat` (Linux) o abrirlo en el editor. Confirmar que tiene 3 líneas.
2. Abrir `archivos1.cpp`, leerlo en voz alta línea por línea y preguntar antes de ejecutar: “¿qué esperan que imprima?”
3. Compilar y ejecutar. Confirmar la salida.
4. Preguntar: “¿qué pasaría si `canciones.txt` no existiera?” Cambiar el nombre del archivo a algo que no existe y mostrar que el programa termina sin imprimir nada ni dar error visible.
5. Agregar la verificación `if (!ifile)` justo después de abrir el archivo y repetir. Ahora sí se muestra el mensaje de error. Esta es la forma correcta de manejar archivos que pueden no existir.
6. Pedir a un alumno que explique qué hace la condición del `while (getline(...))`. Respuesta esperada: `getline` devuelve el stream; cuando llega al EOF el stream es falso.

- **archivos2.cpp: escribir en un archivo**

1. Antes de ejecutar, preguntar: “¿dónde va a aparecer la salida?” Respuesta esperada: en `resultados.txt`, no en pantalla.
2. Ejecutar. Abrir `resultados.txt` con `cat` y mostrar el contenido.
3. Preguntar: “¿cuántos bytes tiene ese archivo?” `ls -lh resultados.txt` o abrir en el hex editor. El archivo tiene `30 10 15\n = 9 caracteres = 9 bytes` (o 10 en Windows por el `\r`).
4. Ejecutar el programa *otra vez* sin cambiar nada. Mostrar que `resultados.txt` sigue igual: `ofstream` sobreescribe. Introducir `fstream::app` y demostrar que ahora agrega una segunda línea.
5. Preguntar: “¿qué pasa si no llamamos a `close()`?” En la mayoría de los casos el destructor del objeto lo cierra automáticamente al salir del `main`,

pero es mala práctica depender de eso, especialmente en programas largos donde el archivo se puede necesitar antes de que termine el programa.

- **archivos3.cpp: leer valores numéricos**

1. Mostrar que este programa lee lo que **archivos2** escribió. Enfatizar: comunicación entre programas a través del archivo.
2. Antes de ejecutar: “¿cuántos » necesitamos para leer los tres números? ¿Pueden ir en la misma línea?” Mostrar ambas formas (separada y encadenada) y confirmar que producen el mismo resultado.
3. Ejecutar y verificar la salida.
4. Pregunta de profundidad: “Si el archivo tuviera más de 3 números, ¿qué pasaría?” Respuesta: los » leen solo los primeros 3; el resto queda sin leer en el stream. “¿Y si tuviera menos de 3?” Los » fallan silenciosamente y `x`, `y` o `z` quedan con valores basura. Para detectarlo se puede verificar el estado del stream: `if (!fresultados)` después de las lecturas.
5. Conectar con la actividad: “El programa que van a escribir en Parte B lee datos estructurados de un archivo. Ese es exactamente el patrón de **archivos3**.”

¿Por qué `getline` y no » para leer canciones?

» lee hasta el primer espacio o salto de línea. “Walking on Sunshine” tiene espacios, así que » leería solo “Walking”. `getline` lee hasta el salto de línea, incluyendo los espacios. Regla práctica: si los datos pueden contener espacios, usa `getline`; si son tokens separados por espacio, usa ».

¿Se puede mezclar `getline` y » en el mismo stream?

Sí, pero con cuidado. » deja el `\n` en el buffer; el siguiente `getline` lo consumirá como una línea vacía. Si vas a mezclarlos, agrega `stream.ignore()` o `stream.ignore(1000, '\n')` después de cada bloque de » para limpiar el buffer antes de `getline`. En la actividad de hoy se evita este problema usando `getline` para cadenas y » para números en campos separados.

¿Qué es EOF exactamente?

End Of File. No es un byte especial dentro del archivo; es una condición que el sistema operativo reporta cuando el stream llega al final de los datos. En el contexto del teclado, `Ctrl+D` (Linux) o `Ctrl+Z` (Windows) simula un EOF.

¿`endl` vs `"\n"` al escribir en archivo?

`endl` escribe `\n` y vacía el buffer (*flush*). `"\n"` solo escribe el carácter. Para archivos, `"\n"` es más rápido si escribes muchas líneas. `endl` es más seguro si necesitas garantizar que el dato llegó al disco inmediatamente (por ejemplo,

en un log de errores).

Actividad: agenda de cumpleaños

Antes de la clase:

- Crear la carpeta en Google Drive y tener el link listo.
- Preparar un archivo `archivos.txt` de ejemplo (vacío al inicio, se llena durante la Parte A) que los alumnos puedan usar para iterar sobre todos los archivos individuales.

Parte A (10 min) — Cada alumno genera su archivo:

1. Dar el link de Drive.
2. Los alumnos escriben un programa con `ofstream` que genere su archivo personal. El formato es cuatro líneas: nombre, apellidos, día, mes.
3. Suben el archivo generado (no el `.cpp`) a Drive.
4. Mientras suben, actualizar `archivos.txt` en Drive con la lista de nombres de archivo para que todos puedan descargarlo.

Parte B (25 min) — Las consultas:

Dar tiempo libre. Circular por el salón para desbloquear a quienes estén atascados. Los puntos de fricción más comunes son:

- Consulta 1: cómo leer un archivo cuyo nombre viene de otro archivo (abrir un `ifstream` con un `string` como nombre).
- Consulta 3: cómo manejar el “año circular” (si ya pasó la fecha este año, buscar la más cercana para el año siguiente).
- Consulta 4: cómo representar la “distancia” entre dos fechas para poder compararlas.

Cierre (5 min): Pedir a 2–3 equipos que muestren su solución a una de las consultas. Elegir preferentemente la 3 o la 4, que tienen mayor riqueza algorítmica.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main(){
```



```

ifstream lista("archivos.txt");
ofstream agenda("agenda.txt");
string archivo;

while(getline(lista, archivo)){
    ifstream f(archivo);
    string nombre, apellido;
    int dia, mes;
    getline(f, nombre);
    getline(f, apellido);
    f >> dia >> mes;
    agenda << nombre << " " << apellido
        << " " << dia << " " << mes << endl;
    f.close();
}
agenda.close();
}

```

Nota: `ifstream f(archivo)` acepta un `string` como nombre desde C++11. En compiladores más antiguos puede requerirse `f.open(archivo.c_str())`.

```

#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main(){
    int mes_buscado;
    cout << "Mes (1-12): ";
    cin >> mes_buscado;

    ifstream agenda("agenda.txt");
    string nombre, apellido;
    int dia, mes;

    while(agenda >> nombre >> apellido >> dia >> mes){
        if(mes == mes_buscado)
            cout << nombre << " " << apellido << endl;
    }
}

```

La idea es convertir cada cumpleaños a “día del año” y comparar con el día actual. Se puede usar $\text{mes} \times 31 + \text{día}$ como aproximación suficiente para encontrar el

mínimo; o calcular el día del año real acumulando días por mes.

Estrategia simple con día del año aproximado:

```
// dias_aprox: aproximación suficiente para ordenar
int dias_aprox(int dia, int mes){
    int acum[] = {0,31,59,90,120,151,181,212,243,273,304,334};
    return acum[mes-1] + dia;
}
```

```
// HOY = 6 de mayo = día ~126
int hoy = dias_aprox(6, 5);
```

```
// Para cada persona calcular dias_aprox(dia, mes)
// Si > hoy: candidato de este año
// Si <= hoy: candidato del año siguiente (sumar 365)
// El próximo cumpleaños es el candidato mínimo
```

En clase es válido que los alumnos elijan la aproximación $\text{mes} \times 31 + \text{día}$ si les funciona para ordenar. El objetivo es algorítmico, no calendárico.

Convertir todas las fechas a días del año (misma función que consulta 3), ordenarlas de menor a mayor, y buscar la diferencia mínima entre elementos consecutivos. No olvidar la diferencia circular entre el último y el primero (suma 365 al último si es menor que el primero).

```
// Con N personas y arreglos nombre[], dia_año[]
// Ordenar por dia_año (bubble sort o cualquier método conocido)
// Luego:
int min_diff = 365;
int p1 = 0, p2 = 1;
for(int i = 0; i < N-1; i++){
    int diff = dia_año[i+1] - dia_año[i];
    if(diff < min_diff){
        min_diff = diff;
        p1 = i; p2 = i+1;
    }
}
// Diferencia circular:
int diff_circular = 365 - dia_año[N-1] + dia_año[0];
if(diff_circular < min_diff){
    min_diff = diff_circular;
    p1 = N-1; p2 = 0;
}
```

Errores frecuentes

1. Archivo no encontrado — sin mensaje de error.

Síntoma: el programa termina sin hacer nada. No hay excepción por defecto.

Remedio: siempre verificar `if (!ifile)` justo después de abrir.

2. Ejecutable en directorio distinto al archivo de datos.

Síntoma: mismo error que el anterior.

Remedio: usar una ruta relativa correcta o ejecutar el programa desde el directorio donde están los datos. En VS Code el directorio de trabajo del ejecutable puede ser distinto al del `.cpp`.

3. Mezclar `>>` y `getline` sin limpiar el buffer.

Síntoma: `getline` lee una cadena vacía en la siguiente llamada.

Remedio: agregar `stream.ignore()` o `stream.ignore(1000, '\n')` después de cada bloque de `>>`.

4. Olvidar `close()` en `ofstream`.

Síntoma: el archivo existe pero está vacío o incompleto. El buffer no se vació.

Remedio: llamar `close()` explícitamente, o mejor aún, envolver el `ofstream` en un bloque `{}` para que su destructor lo cierre.

5. Abrir un `ofstream` para leer (o viceversa).

Síntoma: error de compilación poco claro, o el archivo se borra al abrirlo con `ofstream` (que crea/sobreescribe).

Remedio: `i` = input (leer), `o` = output (escribir).

Soluciones a los ejercicios

```
#include <iostream>
#include <fstream>
using namespace std;

int main(){
    ofstream fout("calificaciones.txt");
    float c1, c2, c3;
    cout << "Tres calificaciones: ";
    cin >> c1 >> c2 >> c3;
    fout << c1 << endl << c2 << endl << c3 << endl;
    fout.close();

    ifstream fin("calificaciones.txt");
    float x, sum = 0;
    int n = 0;
    while(fin >> x){ sum += x; n++; }
```

```
    cout << "Promedio: " << sum/n << endl;
}
```

Solo cambia la línea de apertura de `ofstream`:

```
ofstream fout("calificaciones.txt", fstream::app);
```

El bloque de lectura y promedio queda exactamente igual que en el ejercicio 1. Al acumular todas las calificaciones del archivo completo el promedio ya incluye las anteriores.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main(){
    ifstream fin("canciones.txt");
    ofstream fout("canciones_numeradas.txt");
    string linea;
    int num = 1;
    while(getline(fin, linea)){
        fout << num << ". " << linea << endl;
        num++;
    }
}
```

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main(){
    string a, b, c;
    cout << "Primer archivo: "; cin >> a;
    cout << "Segundo archivo: "; cin >> b;
    cout << "Archivo resultado: "; cin >> c;

    ifstream f1(a), f2(b);
    ofstream fout(c);
    string linea;
    while(getline(f1, linea)) fout << linea << endl;
```

```
    while(getline(f2, linea)) fout << linea << endl;
}
```

Al abrir un `ifstream` con un nombre de archivo que no existe:

- No hay excepción por defecto.
- No hay error de compilación.
- El stream queda en estado de error (`failbit` activado).
- Cualquier operación de lectura sobre él no hace nada.
- `getline` devuelve inmediatamente `false` → el `while` no itera.
- » no modifica la variable de destino (queda con su valor anterior, normalmente basura).

Para detectar el error: `if (!fin)` o `if (fin.fail())` justo después de abrir.
Para activar excepciones explícitas:

```
fin.exceptions(ifstream::failbit | ifstream::badbit);
```

Después de eso, abrir un archivo que no existe lanza `std::ios_base::failure`.

Rúbrica de la actividad

Criterio	Puntos	Evidencia
<i>Parte A — Generar archivo personal</i>		
Archivo con 4 líneas en el formato correcto	5	archivo en Drive
Generado desde un programa C++ con ofstream	5	código fuente
<i>Consulta 1 — Unir archivos</i>		
Lee correctamente cada archivo individual	5	agenda.txt generada
agenda.txt tiene una persona por línea	5	formato correcto
<i>Consulta 2 — Filtrar por mes</i>		
Lee agenda.txt con ifstream	5	código fuente
Muestra solo las personas del mes pedido	5	salida correcta
<i>Consulta 3 — Próximo cumpleaños</i>		
Maneja correctamente el caso “ya pasó este año”	5	prueba con fecha pasada
Muestra todos los compañeros si coinciden	5	prueba con empate
<i>Consulta 4 — Par más cercano</i>		
Resultado correcto para el conjunto completo	5	salida verificada
Maneja el caso circular (dic-ene)	5	prueba con fechas extremas
Total	50	

Conexiones con temas posteriores

- **Sesión 22 — Archivos binarios:** el mismo `<fstream>`, pero con el flag `fstream::binary` y los métodos `write/read`. Los alumnos ya entienden el modelo de stream; solo cambia cómo se serializan los datos.
- **Persistencia en POO:** en el curso de Orientación a Objetos los objetos se guardarán y recuperarán de archivos. El patrón `write/read` que ven hoy es la base de toda serialización.
- **Bases de datos:** un DBMS como SQLite es, al final, un gestor de archivos muy sofisticado. Las operaciones CRUD (Create, Read, Update, Delete) mapean directamente a lo que hicieron hoy con `ofstream` e `ifstream`.

Materiales y recursos

- **Notas alumno:** `notas_alumno/pdf/sesion21_archivos_texto.pdf`
- **Material inicial:** `materialInicial/sesion21/` — `archivos1.cpp`, `archivos2.cpp`, `archivos3.cpp`, `canciones.txt`, `resultados.txt`

- **Hex editor en línea:** hexed.it
- **Google Drive:** carpeta compartida para la actividad (crear antes de la sesión y tener el link listo)